

U N I V E R S I D A D A L E X A N D E R V O N
H U M B O L D T

Facultad de Ingeniería

TypeScript 2026

Guía Completa y Didáctica

Del Ecosistema a la Práctica Profesional

Tutorial para Principiantes y Nivel Intermedio

Gestor de paquetes: Bun | Runtime moderno

Edición 2026

Prólogo

Bienvenido a la guía más completa y didáctica de TypeScript en su versión 2026. Este libro de texto ha sido diseñado especialmente para estudiantes de la Universidad Alexander von Humboldt que desean dominar TypeScript desde los fundamentos hasta un nivel profesional.

TypeScript se ha consolidado como el estándar de facto para el desarrollo de aplicaciones modernas. En 2026, el ecosistema ha madurado enormemente: Bun se ha posicionado como el gestor de paquetes y runtime preferido, el sistema de tipos ha alcanzado niveles de expresividad sin precedentes, y las herramientas de desarrollo ofrecen experiencias de productividad inigualables.

Este tutorial adopta un enfoque para dummies: cada concepto se explica paso a paso, con analogías del mundo real, ejemplos prácticos comentados línea por línea y ejercicios de aplicación. No se asume ningún conocimiento previo de TypeScript, aunque sí se espera familiaridad básica con lógica de programación.

Convenciones de este libro: Los bloques de código aparecen sombreados en gris claro con tipografía monoespaciada. Los recuadros verdes contienen consejos prácticos. Los recuadros azules son notas conceptuales. Los recuadros naranjas son advertencias importantes.

✓ Consejo didáctico

Antes de leer cada capítulo, intenta responder: ¿Qué creo que significa este concepto? Luego compara tu respuesta con la explicación del libro. Este método activa tu aprendizaje significativo.

Índice de Contenidos

Prólogo	2
Índice de Contenidos.....	3
Capítulo 1: El Ecosistema TypeScript en 2026.....	6
1.1 ¿Qué es TypeScript?	6
1.2 ¿Por qué TypeScript en 2026?	6
1.3 Bun: El Runtime Moderno	6
1.3.1 Instalación de Bun	7
1.3.2 Crear un proyecto TypeScript con Bun	7
1.3.3 Ejecutar archivos TypeScript	7
1.4 Estructura del archivo tsconfig.json	8
1.5 Herramientas del Ecosistema 2026.....	8
Capítulo 2: Sistema de Tipado	10
2.1 ¿Qué es el Tipado Estático?.....	10
2.2 Tipos Primitivos	10
2.3 Inferencia de Tipos	11
2.4 Tipos Especiales.....	11
2.4.1 any, unknown, never y void	11
2.5 Arrays y Tuplas	12
2.6 Type Aliases y Union Types	13
2.7 Literal Types e Intersection Types	13
Capítulo 3: Operadores	15
3.1 Operadores Aritméticos	15
3.2 Operadores de Comparación	15
3.3 Operadores Lógicos	16
3.4 Operadores Especiales de TypeScript.....	16
3.4.1 Optional Chaining (?.)	16
3.4.2 Nullish Coalescing (??).....	17
3.4.3 Non-null Assertion (!)	17
3.4.4 Operador satisfies (TypeScript 5.x+).....	17
3.4.5 Operador typeof (Type Guard).....	18
Capítulo 4: Funciones	19
4.1 Fundamentos de Funciones en TypeScript.....	19
4.2 Declaración de Funciones.....	19

4.3 Parámetros Opcionales y con Valor por Defecto	19
4.4 Parámetros Rest (Variadic).....	20
4.5 Sobrecarga de Funciones (Overloads)	20
4.6 Funciones como Tipos	20
Capítulo 5: Funciones Flecha (Arrow Functions)	22
5.1 Sintaxis de Funciones Flecha	22
5.2 Diferencias con Funciones Tradicionales.....	22
5.3 El Problema del this.....	22
5.4 Patrones Comunes con Funciones Flecha	23
5.5 Funciones Flecha Genéricas	24
Capítulo 6: Estructuras de Control	25
6.1 Condicionales	25
6.1.1 if / else if / else	25
6.1.2 Operador Ternario.....	25
6.1.3 switch con Exhaustive Checking	25
6.2 Bucles	26
6.2.1 for clásico	26
6.2.2 for...of (iterar valores)	26
6.2.3 for...in (iterar claves)	27
6.2.4 while y do...while.....	27
6.3 Control de Flujo Avanzado	27
6.3.1 Discriminated Unions (Pattern Matching).....	27
Capítulo 7: Clases	29
7.1 Fundamentos de Clases.....	29
7.2 Declaración de Clases con Tipos.....	29
7.3 Modificadores de Acceso	29
7.4 Sintaxis Abreviada del Constructor	30
7.5 Herencia	31
7.6 Clases Abstractas.....	32
Capítulo 8: Interfaces	34
8.1 ¿Qué es una Interface?.....	34
8.2 Declaración de Interfaces.....	34
8.3 Interfaces con Métodos.....	34
8.4 Extensión de Interfaces	35
8.5 Interfaces vs Type Aliases.....	36

8.6 Implementación de Interfaces en Clases.....	36
Capítulo 9: Temas Complementarios Avanzados	38
9.1 Genéricos (Generics)	38
9.1.1 Restricciones en Genéricos (Constraints)	39
9.2 Módulos e Importaciones.....	39
9.3 Tipos de Utilidad (Utility Types)	40
9.4 Enums	41
9.5 Manejo de Errores Tipado	41
9.6 Programación Asíncrona Tipada	42
9.7 Decoradores (TypeScript 5.x+ Estándar)	43
9.8 Validación con Zod	43
Capítulo 10: Proyecto Práctico Integrador	45
10.1 Descripción del Proyecto	45
10.2 Definición de Tipos e Interfaces	45
10.3 Clase Genérica de Repositorio.....	46
10.4 Servicio Académico	47
10.5 Ejecución del Proyecto	48
Glosario de Términos	49
Bibliografía y Recursos	50

Capítulo 1: El Ecosistema TypeScript en 2026

1.1 ¿Qué es TypeScript?

TypeScript es un lenguaje de programación de código abierto desarrollado y mantenido por Microsoft. Técnicamente, es un superconjunto tipado de JavaScript: esto significa que todo código JavaScript válido también es código TypeScript válido, pero TypeScript agrega características adicionales, siendo la más importante el sistema de tipos estáticos.

Analogía: Imagina que JavaScript es como hablar un idioma sin reglas gramaticales: puedes comunicarte, pero es fácil cometer errores que nadie detecta hasta que alguien se confunde. TypeScript es como agregar gramática formal al idioma: el compilador actúa como un corrector que te avisa de errores antes de que tu mensaje llegue al destinatario.

1.2 ¿Por qué TypeScript en 2026?

En 2026, TypeScript ha superado la versión 5.7+ y se ha integrado de forma nativa con los principales runtimes. Las razones para aprenderlo hoy son contundentes:

Razón	Descripción
Seguridad de tipos	Detecta errores en tiempo de compilación, no en producción
Productividad	Autocompletado inteligente, refactorización segura y documentación inline
Ecosistema maduro	Más del 90% de librerías populares incluyen tipos nativos
Demanda laboral	Es el lenguaje más solicitado para desarrollo fullstack
Soporte nativo de Bun	Ejecución directa de .ts sin compilación previa
IA y herramientas	Los modelos de IA generan mejor código TypeScript tipado

1.3 Bun: El Runtime Moderno

Bun es un runtime de JavaScript y TypeScript escrito en Zig, diseñado para ser extremadamente rápido. En 2026, Bun ha reemplazado en gran medida a Node.js para nuevos proyectos gracias a sus ventajas:

Característica	Node.js	Bun
Ejecución de TypeScript	Requiere ts-node o compilación	Nativo, sin configuración
Gestor de paquetes	npm (separado)	Integrado (bun install)

Característica	Node.js	Bun
Velocidad de instalación	Moderada	Hasta 25x más rápido
Test runner	Jest/Vitest (externo)	Integrado (bun test)
Bundler	Webpack/Vite (externo)	Integrado (bun build)
Hot reload	nodemon (externo)	Integrado (bun --watch)

1.3.1 Instalación de Bun

La instalación de Bun es sencilla en cualquier sistema operativo:

```
# macOS / Linux
curl -fsSL https://bun.sh/install | bash

# Windows (PowerShell)
irm bun.sh/install.ps1 | iex

# Verificar instalación
bun --version
```

1.3.2 Crear un proyecto TypeScript con Bun

Bun simplifica la creación de proyectos TypeScript a un solo comando:

```
# Inicializar un proyecto nuevo
bun init

# Esto genera automáticamente:
# └─ package.json
# └─ tsconfig.json   ← Configuración de TypeScript
# └─ index.ts        ← Archivo principal
# └─ bun.lockb       ← Archivo de bloqueo de dependencias
```

1.3.3 Ejecutar archivos TypeScript

A diferencia de Node.js, Bun ejecuta TypeScript directamente sin compilación previa:

```
# Ejecutar directamente un archivo .ts
bun run index.ts

# Ejecutar con hot reload (recarga automática)
bun --watch index.ts

# Ejecutar en modo producción
```

```
bun run --production index.ts
```

✓ Consejo práctico

Usa siempre `bun --watch` durante el desarrollo. Cada vez que guardes un cambio en tu archivo, Bun reiniciará automáticamente la ejecución, ahorrando tiempo valioso.

1.4 Estructura del archivo `tsconfig.json`

El archivo `tsconfig.json` es el corazón de la configuración de TypeScript. Define cómo se comporta el compilador y qué reglas aplica a tu código:

```
{
  "compilerOptions": {
    "target": "ESNext",           // Versión de JS destino
    "module": "ESNext",          // Sistema de módulos
    "moduleResolution": "bundler", // Resolución moderna
    "strict": true,               // Todas las reglas estrictas
    "noUncheckedIndexedAccess": true, // Acceso seguro a arrays
    "exactOptionalPropertyTypes": true,
    "outDir": "./dist",           // Carpeta de salida
    "rootDir": "./src",           // Carpeta fuente
    "declaration": true,          // Generar archivos .d.ts
    "sourceMap": true              // Mapas de depuración
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Nota conceptual

La opción `"strict": true` activa un conjunto de verificaciones que garantizan código más seguro. En 2026, esta opción es obligatoria en cualquier proyecto profesional. Nunca la desactives.

1.5 Herramientas del Ecosistema 2026

El ecosistema moderno de TypeScript incluye herramientas especializadas que potencian la productividad:

Herramienta	Propósito	Comando
Bun	Runtime + gestor de paquetes	<code>bun run / bun install</code>

Herramienta	Propósito	Comando
Biome	Linter + Formatter (reemplaza ESLint + Prettier)	biome check / biome format
Vitest	Framework de testing compatible con Bun	bun test
Drizzle ORM	ORM tipado para bases de datos	drizzle-kit generate
Zod	Validación de esquemas con inferencia de tipos	import { z } from 'zod'
tRPC	APIs tipadas end-to-end	import { router } from '@trpc/server'
Hono	Framework web ultraligero para Bun	new Hono()

Capítulo 2: Sistema de Tipado

2.1 ¿Qué es el Tipado Estático?

El tipado estático significa que cada variable, parámetro y valor de retorno tiene un tipo definido que se verifica en tiempo de compilación, antes de ejecutar el programa. Esto contrasta con JavaScript, donde los tipos se verifican en tiempo de ejecución (tipado dinámico).

Analogía: El tipado estático es como etiquetar cada caja en una mudanza antes de cargar el camión. Si intentas meter un plato en una caja etiquetada como "libros", el sistema te avisa del error antes de que se rompa algo. En JavaScript, cargas las cajas sin etiquetar y descubres los problemas cuando llegas a destino.

2.2 Tipos Primitivos

TypeScript ofrece tipos primitivos que representan los valores más básicos del lenguaje:

```
// String: texto
let nombre: string = "María";
let saludo: string = `Hola, ${nombre}`; // Template literal

// Number: números enteros y decimales
let edad: number = 25;
let precio: number = 99.99;
let hexadecimal: number = 0xFF;

// Boolean: verdadero o falso
let activo: boolean = true;
let eliminado: boolean = false;

// BigInt: números muy grandes (ES2020+)
let distanciaEstelar: bigint = 946100000000000n;

// Symbol: identificador único
let id: symbol = Symbol("identificador");

// Null y Undefined
let vacio: null = null;
let indefinido: undefined = undefined;
```

✓ Consejo práctico

En la mayoría de los casos, TypeScript puede inferir el tipo automáticamente. Escribe `let nombre = "María"` y TypeScript sabrá que es `string`. Usa anotaciones explícitas solo cuando la inferencia no sea obvia o para parámetros de funciones.

2.3 Inferencia de Tipos

TypeScript posee un motor de inferencia sofisticado que deduce automáticamente el tipo de una variable a partir de su valor asignado:

```
// TypeScript INFIERE los tipos automáticamente:
let ciudad = "Bogotá";           // Tipo inferido: string
let poblacion = 8000000;         // Tipo inferido: number
let esCapital = true;            // Tipo inferido: boolean

// La inferencia también funciona con expresiones complejas:
let resultado = 10 + 20;          // Tipo inferido: number
let mensaje = `Hola ${ciudad}`;  // Tipo inferido: string

// ¡TypeScript detecta errores gracias a la inferencia!
ciudad = 42;                      // ❌ Error: Type "number" is not assignable to type "string"
```

2.4 Tipos Especiales

2.4.1 any, unknown, never y void

TypeScript incluye tipos especiales que cubren escenarios particulares. Comprenderlos es fundamental para escribir código seguro:

Tipo	Significado	¿Cuándo usarlo?
any	Acepta cualquier tipo, desactiva verificación	NUNCA en código profesional (solo migraciones)
unknown	Acepta cualquier tipo, pero exige verificación	Cuando recibes datos de origen desconocido
never	Ningún valor posible	Funciones que lanzan errores o bucles infinitos
void	Sin valor de retorno	Funciones que no retornan nada

```
// ✅ unknown: seguro, requiere verificación antes de usar
function procesarDato(dato: unknown): string {
  if (typeof dato === "string") {
```

```
    return dato.toUpperCase(); // TypeScript sabe que es string aquí
  }
  if (typeof dato === "number") {
    return dato.toFixed(2); // TypeScript sabe que es number aquí
  }
  return "Tipo no soportado";
}

// ✅ never: la función nunca retorna normalmente
function lanzarError(mensaje: string): never {
  throw new Error(mensaje);
}

// ✅ void: la función no retorna un valor
function saludar(nombre: string): void {
  console.log(`Hola, ${nombre}`);
}
```

⚠️ Advertencia

Evita el tipo `any` a toda costa. En 2026, usar `any` es considerado una mala práctica grave. Si no conoces el tipo, usa `unknown` y realiza verificaciones de tipo (type guards) antes de operar con el valor.

2.5 Arrays y Tuplas

Los arrays en TypeScript son colecciones tipadas de elementos. Las tuplas son arrays con un número fijo de elementos donde cada posición tiene un tipo específico:

```
// Arrays: todos los elementos del mismo tipo
let numeros: number[] = [1, 2, 3, 4, 5];
let nombres: Array<string> = ["Ana", "Luis", "Carlos"];

// Arrays de solo lectura (inmutables)
let constantes: readonly number[] = [3.14, 2.71, 1.61];
// constantes.push(42); ❌ Error: propiedad push no existe

// Tuplas: longitud y tipos fijos por posición
let coordenada: [number, number] = [4.60, -74.08];
let persona: [string, number, boolean] = ["Ana", 28, true];

// Tuplas con etiquetas (TypeScript 5.x+, legibles y documentadas)
let ubicacion: [latitud: number, longitud: number] = [4.60, -74.08];

// Acceso tipado a elementos de tupla
let lat = ubicacion[0]; // Tipo: number
let lon = ubicacion[1]; // Tipo: number
```

2.6 Type Aliases y Union Types

Los alias de tipo permiten crear nombres descriptivos para tipos complejos, mientras que los tipos unión permiten que una variable acepte múltiples tipos:

```
// Type Alias: crear un nombre para un tipo
type Edad = number;
type Nombre = string;
type EstadoCivil = "soltero" | "casado" | "divorciado" | "viudo";

// Union Types: aceptar múltiples tipos
type ID = string | number;

let usuarioId: ID = "abc-123"; // ✅ Válido
usuarioId = 456; // ✅ Válido
// usuarioId = true; // ❌ Error: boolean no assignable a ID

// Objetos con Type Alias
type Estudiante = {
  nombre: string;
  edad: number;
  carrera: string;
  matricula: string;
  promedio?: number; // ? = propiedad opcional
};

const alumno: Estudiante = {
  nombre: "Carlos Pérez",
  edad: 22,
  carrera: "Ingeniería de Sistemas",
  matricula: "2024-001",
  // promedio es opcional, no es necesario incluirlo
};
```

2.7 Literal Types e Intersection Types

Los tipos literales restringen una variable a valores específicos, y los tipos intersección combinan múltiples tipos en uno:

```
// Literal Types: valores exactos como tipos
type Direccion = "norte" | "sur" | "este" | "oeste";
let rumbo: Direccion = "norte"; // ✅
// rumbo = "noroeste"; // ❌ Error
```

```
typeCodigoHTTP = 200 | 201 | 400 | 404 | 500;
let status: CodigoHTTP = 200;      // 

// Intersection Types: combinar tipos con &
type ConDireccion = { direccion: string; ciudad: string };
type ConTelefono = { telefono: string; extension?: number };

type Contacto = ConDireccion & ConTelefono;

const contacto: Contacto = {
  direccion: "Calle 123 #45-67",
  ciudad: "Bogotá",
  telefono: "+57 300 123 4567",
  extension: 101,
};
```

Capítulo 3: Operadores

3.1 Operadores Aritméticos

Los operadores aritméticos realizan cálculos matemáticos sobre valores numéricos. TypeScript hereda todos los operadores de JavaScript y añade seguridad de tipos:

```
let a: number = 20;
let b: number = 7;

let suma      = a + b;    // 27
let resta     = a - b;    // 13
let multiplicacion = a * b; // 140
let division  = a / b;    // 2.857...
let modulo    = a % b;    // 6 (residuo)
let exponente = a ** 2;   // 400 (potencia)

// Operadores de asignación compuesta
let x = 10;
x += 5;    // x = x + 5 → 15
x -= 3;    // x = x - 3 → 12
x *= 2;    // x = x * 2 → 24
x /= 4;    // x = x / 4 → 6
x **= 3;   // x = x ^ 3 → 216
```

3.2 Operadores de Comparación

Los operadores de comparación comparan dos valores y devuelven un boolean. En TypeScript se recomienda siempre usar la comparación estricta (=== y !==):

Operador	Significado	Ejemplo	Resultado
===	Igualdad estricta (tipo + valor)	5 === 5	true
!==	Desigualdad estricta	5 !== "5"	true
>	Mayor que	10 > 5	true
<	Menor que	3 < 8	true
>=	Mayor o igual que	5 >= 5	true
<=	Menor o igual que	4 <= 3	false

 **Advertencia**

NUNCA uses `==` o `!=` en TypeScript. Estos operadores realizan coerción de tipos y producen resultados inesperados. Siempre usa `===` y `!==`. TypeScript en modo estricto te advertirá si usas comparaciones no estrictas.

3.3 Operadores Lógicos

Los operadores lógicos permiten combinar expresiones booleanas para construir condiciones complejas:

```
let esMayor: boolean = true;
let tienePermiso: boolean = false;

// AND (&&): ambas condiciones deben ser verdaderas
let puedeEntrar = esMayor && tienePermiso; // false

// OR (||): al menos una condición verdadera
let tieneAcceso = esMayor || tienePermiso; // true

// NOT (!): invierte el valor
let estaBloqueado = !tienePermiso; // true

// Ejemplo práctico: validación de formulario
let edad = 20;
let pais = "Colombia";
let aceptoTerminos = true;

let puedeRegistrarse = edad >= 18 && pais === "Colombia" && aceptoTerminos;
console.log(puedeRegistrarse); // true
```

3.4 Operadores Especiales de TypeScript

TypeScript introduce operadores especiales que mejoran la seguridad y legibilidad del código:

3.4.1 Optional Chaining (?.)

Permite acceder a propiedades de objetos que podrían ser null o undefined sin provocar errores en tiempo de ejecución:

```
type Usuario = {
  nombre: string;
  direccion?: {
    calle?: string;
    ciudad: string;
  };
};
```

```
const usuario: Usuario = { nombre: "Ana" };

// Sin optional chaining: error potencial
// const calle = usuario.direccion.calle; ❌ Puede fallar

// Con optional chaining: seguro
const calle = usuario.direccion?.calle; // undefined (sin error)
const ciudad = usuario.direccion?.ciudad; // undefined (sin error)
```

3.4.2 Nullish Coalescing (??)

Proporciona un valor por defecto cuando una expresión es null o undefined (no cuando es 0 o cadena vacía, a diferencia de ||):

```
let nombre: string | null = null;
let mostrar = nombre ?? "Anónimo"; // "Anónimo"

// Diferencia crucial con ||:
let cantidad: number = 0;
let conOR = cantidad || "sin datos"; // "sin datos" ❌ incorrecto
let conNullish = cantidad ?? 100; // 0 ✅ correcto

// ?? solo reacciona a null y undefined, no a 0, false, o ""
```

3.4.3 Non-null Assertion (!)

Le dice al compilador que un valor NO es null ni undefined. Úsalo con precaución:

```
// Cuando ESTÁS SEGURO de que el valor existe
const elemento = document.getElementById("app")!;
// Sin el !, TypeScript marca error porque podría ser null

// ⚠️ Mejor alternativa: validar explícitamente
const elementoSeguro = document.getElementById("app");
if (elementoSeguro) {
  elementoSeguro.textContent = "Cargado";
}
```

3.4.4 Operator satisfies (TypeScript 5.x+)

El operador satisfies verifica que un valor cumple con un tipo sin cambiar el tipo inferido. Es ideal para validar configuraciones:

```
type Colores = Record<string, [number, number, number]>;
```

```
// satisfies valida la estructura pero mantiene la inferencia exacta
const paleta = {
  rojo: [255, 0, 0],
  verde: [0, 255, 0],
  azul: [0, 0, 255],
} satisfies Colores;

// TypeScript sabe exactamente qué colores existen:
const r = paleta.rojo;    // Tipo: [number, number, number]
// paleta.amarillo;      // ❌ Error: la propiedad no existe
```

3.4.5 Operador typeof (Type Guard)

typeof se usa como guardia de tipos para refinar el tipo de una variable dentro de un bloque condicional:

```
function formatear(valor: string | number): string {
  if (typeof valor === "string") {
    // Aquí TypeScript sabe que valor es string
    return valor.toUpperCase();
  }
  // Aquí TypeScript sabe que valor es number
  return valor.toFixed(2);
}

console.log(formatear("hola")); // "HOLA"
console.log(formatear(3.14159)); // "3.14"
```

Capítulo 4: Funciones

4.1 Fundamentos de Funciones en TypeScript

Las funciones son los bloques constructores fundamentales de cualquier programa. En TypeScript, las funciones incluyen tipos para sus parámetros y valor de retorno, lo que garantiza que se usen correctamente en todo el código:

Analogía: Una función es como una máquina expendedora: insertas una moneda (parámetro de entrada con un tipo específico) y recibes un producto (valor de retorno con un tipo específico). TypeScript se asegura de que solo insertes monedas válidas y de que siempre recibas el producto esperado.

4.2 Declaración de Funciones

```
// Función con tipos explícitos de parámetros y retorno
function sumar(a: number, b: number): number {
    return a + b;
}

// TypeScript verifica los tipos al llamar la función:
const resultado = sumar(5, 3);           // ✅ resultado: number
// const error = sumar("5", 3);         // ❌ Error de tipo
// const error2 = sumar(5, 3, 1);       // ❌ Demasiados argumentos
```

4.3 Parámetros Opcionales y con Valor por Defecto

```
// Parámetro opcional: se indica con ?
function saludar(nombre: string, titulo?: string): string {
    if (titulo) {
        return `Hola, ${titulo} ${nombre}`;
    }
    return `Hola, ${nombre}`;
}

saludar("Carlos");           // "Hola, Carlos"
saludar("Carlos", "Dr.");    // "Hola, Dr. Carlos"

// Parámetro con valor por defecto
function crearUsuario(nombre: string, rol: string = "estudiante"): string {
    return `${nombre} (${rol})`;
}

crearUsuario("Ana");         // "Ana (estudiante)"
```

```
crearUsuario("Ana", "docente"); // "Ana (docente)"
```

4.4 Parámetros Rest (Variadic)

Los parámetros rest permiten que una función acepte un número indefinido de argumentos:

```
// ...numeros captura todos los argumentos como un array
function sumarTodos(...numeros: number[]): number {
  return numeros.reduce((total, num) => total + num, 0);
}

sumarTodos(1, 2, 3);           // 6
sumarTodos(10, 20, 30, 40);    // 100

// Combinar parámetros fijos con rest
function registrar(accion: string, ...detalles: string[]): void {
  console.log(`[${accion}] ${detalles.join(", ")}`);
}

registrar("LOGIN", "usuario: ana", "ip: 192.168.1.1");
```

4.5 Sobrecarga de Funciones (Overloads)

La sobrecarga permite que una función tenga múltiples firmas con diferentes tipos de parámetros y retorno:

```
// Firmas de sobrecarga (solo declaración de tipos)
function buscar(id: number): Usuario;
function buscar(email: string): Usuario;
function buscar(criterio: { nombre: string; edad: number }): Usuario[];

// Implementación (debe ser compatible con todas las firmas)
function buscar(criterio: number | string | { nombre: string; edad: number }):
  Usuario | Usuario[] {
  if (typeof criterio === "number") {
    return obtenerPorId(criterio);
  } else if (typeof criterio === "string") {
    return obtenerPorEmail(criterio);
  } else {
    return filtrarPorCriterio(criterio);
  }
}
```

4.6 Funciones como Tipos

En TypeScript, las funciones son ciudadanos de primera clase. Puedes definir tipos para funciones:

```
// Definir un tipo de función
type Operacion = (a: number, b: number) => number;

// Usar el tipo para garantizar consistencia
const sumar: Operacion = (a, b) => a + b;
const restar: Operacion = (a, b) => a - b;
const multiplicar: Operacion = (a, b) => a * b;

// Función que recibe otra función como parámetro
function calcular(a: number, b: number, operacion: Operacion): number {
  return operacion(a, b);
}

calcular(10, 5, sumar);          // 15
calcular(10, 5, restar);        // 5
calcular(10, 5, multiplicar);   // 50
```

Capítulo 5: Funciones Flecha (Arrow Functions)

5.1 Sintaxis de Funciones Flecha

Las funciones flecha son una forma concisa de escribir funciones. Fueron introducidas en ES6 y son el estándar dominante en TypeScript 2026. Su principal ventaja, además de la brevedad, es que heredan el contexto `this` del ámbito donde se definen:

```
// Función tradicional
function duplicar(x: number): number {
  return x * 2;
}

// Equivalente como función flecha
const duplicarFlecha = (x: number): number => {
  return x * 2;
};

// Forma ultra-concisa (retorno implícito, una sola expresión)
const duplicarCorta = (x: number): number => x * 2;

// Si solo hay UN parámetro, los paréntesis son opcionales en JS
// pero en TypeScript se requieren para anotar el tipo
const triplicar = (x: number): number => x * 3;
```

5.2 Diferencias con Funciones Tradicionales

Característica	function tradicional	Arrow function
Contexto this	Propio (dinámico)	Hereda del padre (léxico)
Uso como método	Recomendado	Evitar (this problemático)
Constructor	Sí (con new)	No permite new
arguments	Disponible	No disponible (usar rest)
Sintaxis	Más verbosa	Concisa
Hoisting	Sí (se eleva)	No (debe declararse antes)

5.3 El Problema del this

La razón principal para usar funciones flecha es el manejo predecible de `this`. Observa la diferencia:

```
class Temporizador {
  segundos: number = 0;

  // ❌ Problema con función tradicional
  iniciarMal() {
    setInterval(function() {
      this.segundos++; // ❌ Error: this no apunta a Temporizador
    }, 1000);
  }

  // ✅ Solución con función flecha
  iniciarBien() {
    setInterval(() => {
      this.segundos++; // ✅ this apunta a Temporizador
      console.log(this.segundos);
    }, 1000);
  }
}
```

Nota conceptual

En TypeScript 2026, la regla general es: usa funciones flecha para callbacks, event handlers y funciones inline. Usa function para métodos de clase y funciones que necesiten su propio contexto this.

5.4 Patrones Comunes con Funciones Flecha

```
// Transformar arrays con map
const precios = [100, 200, 300];
const conIVA = precios.map((precio) => precio * 1.19);
// [119, 238, 357]

// Filtrar con filter
const mayores = [15, 22, 17, 30, 12].filter((edad) => edad >= 18);
// [22, 30]

// Reducir con reduce
const total = precios.reduce((acum, precio) => acum + precio, 0);
// 600

// Retorno de objetos (requiere paréntesis)
const crearUsuario = (nombre: string, edad: number) => ({
  nombre,
  edad,
  activo: true,
});
```

```
// Funciones de orden superior (retornan funciones)
const multiplicarPor = (factor: number) => (valor: number) => valor * factor;
const doble = multiplicarPor(2);
const triple = multiplicarPor(3);

doble(5);    // 10
triple(5);   // 15
```

5.5 Funciones Flecha Genéricas

Las funciones flecha también pueden usar genéricos para ser reutilizables con múltiples tipos:

```
// Función genérica que retorna el primer elemento
const primero = <T>(arr: T[]): T | undefined => arr[0];

primero([1, 2, 3]);           // Tipo: number | undefined
primero(["a", "b", "c"]);      // Tipo: string | undefined

// Función genérica con restricción
const obtenerPropiedad = <T, K extends keyof T>(obj: T, key: K): T[K] => {
  return obj[key];
};

const nombre = obtenerPropiedad({ nombre: "Ana", edad: 25 }, "nombre");
// Tipo: string
```

Capítulo 6: Estructuras de Control

6.1 Condicionales

6.1.1 if / else if / else

La estructura if-else permite ejecutar bloques de código según una condición. TypeScript refina los tipos automáticamente dentro de cada rama:

```
function clasificarNota(nota: number): string {
  if (nota >= 4.5) {
    return "Excelente";
  } else if (nota >= 3.5) {
    return "Bueno";
  } else if (nota >= 3.0) {
    return "Aceptable";
  } else {
    return "Reprobado";
  }
}

console.log(clasificarNota(4.7)); // "Excelente"
console.log(clasificarNota(2.8)); // "Reprobado"
```

6.1.2 Operador Ternario

El operador ternario es una forma concisa de escribir un if-else en una sola línea:

```
// Sintaxis: condición ? valorSiVerdadero : valorSiFalso
const edad = 20;
const tipo = edad >= 18 ? "adulto" : "menor";

// Ternarios anidados (usar con moderación)
const categoria = edad >= 65
  ? "senior"
  : edad >= 18
    ? "adulto"
    : "menor";
```

6.1.3 switch con Exhaustive Checking

El switch es especialmente poderoso en TypeScript porque permite verificar que todos los casos de un tipo unión estén cubiertos (exhaustive checking):

```
type Semaforo = "rojo" | "amarillo" | "verde";
```

```
function obtenerAccion(color: Semaforo): string {
  switch (color) {
    case "rojo":
      return "Detenerse";
    case "amarillo":
      return "Precaución";
    case "verde":
      return "Avanzar";
    default:
      // Exhaustive check: si añades un nuevo color al tipo,
      // TypeScript marcará un error AQUÍ, obligándote a manejarlo
      const _exhaustive: never = color;
      return _exhaustive;
  }
}
```

✓ Consejo didáctico

El patrón de exhaustive checking con never es una de las técnicas más valiosas de TypeScript. Garantiza que cada vez que añadas una nueva variante a un tipo unión, el compilador te obligue a manejarla en todos los switch.

6.2 Bucles

6.2.1 for clásico

```
// Bucle for clásico
for (let i = 0; i < 5; i++) {
  console.log(`Iteración ${i}`);
}
```

6.2.2 for...of (iterar valores)

for...of es la forma recomendada para iterar sobre arrays, strings y cualquier iterable:

```
const frutas: string[] = ["manzana", "pera", "uva"];

for (const fruta of frutas) {
  console.log(fruta); // TypeScript sabe que fruta es string
}

// Con desestructuración
const estudiantes: [string, number][] = [["Ana", 4.5], ["Luis", 3.8]];

for (const [nombre, nota] of estudiantes) {
  console.log(`${nombre}: ${nota}`);
}
```

```
}
```

6.2.3 for...in (iterar claves)

```
const configuracion = { tema: "oscuro", idioma: "es", fuente: 14 };

for (const clave in configuracion) {
  console.log(`${clave}: ${configuracion[clave as keyof typeof configuracion]}`);
}
```

6.2.4 while y do...while

```
// while: verifica ANTES de ejecutar
let contador = 0;
while (contador < 3) {
  console.log(contador);
  contador++;
}

// do...while: ejecuta AL MENOS UNA VEZ
let intentos = 0;
do {
  intentos++;
  console.log(`Intento ${intentos}`);
} while (intentos < 3);
```

6.3 Control de Flujo Avanzado

6.3.1 Discriminated Unions (Pattern Matching)

Las discriminated unions combinan tipos unión con una propiedad discriminante para crear patrones de control de flujo extremadamente seguros:

```
// Cada variante tiene una propiedad 'tipo' que las distingue
type Figura =
  | { tipo: "circulo"; radio: number }
  | { tipo: "rectangulo"; ancho: number; alto: number }
  | { tipo: "triangulo"; base: number; altura: number };

function calcularArea(figura: Figura): number {
  switch (figura.tipo) {
    case "circulo":
      return Math.PI * figura.radio ** 2;
    case "rectangulo":
      return figura.ancho * figura.alto;
  }
}
```

```
    case "triangulo":  
        return (figura.base * figura.altura) / 2;  
    }  
}  
  
// TypeScript sabe exactamente qué propiedades tiene cada caso  
calcularArea({ tipo: "circulo", radio: 5 });  
// calcularArea({ tipo: "circulo", ancho: 5 }); ❌ Error
```

Capítulo 7: Clases

7.1 Fundamentos de Clases

Las clases en TypeScript son plantillas para crear objetos que comparten la misma estructura y comportamiento. TypeScript extiende las clases de JavaScript con modificadores de acceso, propiedades tipadas y características avanzadas:

Analogía: Una clase es como el plano de una casa. El plano define la estructura (habitaciones, puertas, ventanas) y el comportamiento (iluminación, calefacción). Cada casa construida a partir del plano es un objeto (instancia). Todas comparten la estructura, pero cada una tiene sus propios valores (color, muebles, habitantes).

7.2 Declaración de Clases con Tipos

```
class Estudiante {
  // Propiedades tipadas
  nombre: string;
  edad: number;
  carrera: string;
  promedio: number;

  // Constructor: inicializa las propiedades
  constructor(nombre: string, edad: number, carrera: string) {
    this.nombre = nombre;
    this.edad = edad;
    this.carrera = carrera;
    this.promedio = 0;
  }

  // Método
  presentarse(): string {
    return `Soy ${this.nombre}, tengo ${this.edad} años ` +
      `y estudio ${this.carrera}`;
  }
}

// Crear una instancia
const alumno = new Estudiante("María", 21, "Ingeniería");
console.log(alumno.presentarse());
```

7.3 Modificadores de Acceso

TypeScript ofrece tres modificadores de acceso que controlan la visibilidad de propiedades y métodos:

Modificador	Accesible desde	Uso común
public	Cualquier lugar (por defecto)	APIs públicas de la clase
private	Solo dentro de la clase	Datos internos, implementación
protected	Clase actual y subclasses	Datos compartidos en herencia
readonly	Solo lectura después de asignar	Constantes de instancia

```

class CuentaBancaria {
  public titular: string;
  private saldo: number;
  protected banco: string;
  readonly numeroCuenta: string;

  constructor(titular: string, saldoInicial: number) {
    this.titular = titular;
    this.saldo = saldoInicial;
    this.banco = "Banco Humboldt";
    this.numeroCuenta = this.generarNumero();
  }

  private generarNumero(): string {
    return `CTA-${Date.now()}`;
  }

  public depositar(monto: number): void {
    if (monto <= 0) throw new Error("Monto inválido");
    this.saldo += monto;
  }

  public consultarSaldo(): number {
    return this.saldo;
  }
}

const cuenta = new CuentaBancaria("Ana", 1000);
cuenta.depositar(500);
console.log(cuenta.consultarSaldo()); // 1500
// cuenta.saldo = 0; ❌ Error: saldo es private

```

7.4 Sintaxis Abreviada del Constructor

TypeScript permite declarar y asignar propiedades directamente en el constructor, reduciendo código repetitivo:

```
// ❌ Forma larga (repetitiva)
class ProductoLargo {
  nombre: string;
  precio: number;
  constructor(nombre: string, precio: number) {
    this.nombre = nombre;
    this.precio = precio;
  }
}

// ✅ Forma abreviada (recomendada en 2026)
class Producto {
  constructor(
    public readonly nombre: string,
    public precio: number,
    private stock: number = 0,
  ) {}

  vender(cantidad: number): boolean {
    if (cantidad > this.stock) return false;
    this.stock -= cantidad;
    return true;
  }
}
```

7.5 Herencia

La herencia permite crear clases especializadas a partir de clases base. TypeScript usa la palabra clave `extends`:

```
// Clase base (padre)
class Animal {
  constructor(
    protected nombre: string,
    protected edad: number,
  ) {}

  presentarse(): string {
    return `Soy ${this.nombre}, tengo ${this.edad} años`;
  }
}

// Clase derivada (hija)
class Perro extends Animal {
  constructor(
    nombre: string,
    edad: number,
  ) {}
}
```

```
    private raza: string,
  ) {
    super(nombre, edad); // Llamar al constructor padre
  }

  // Sobrescribir método del padre
  override presentarse(): string {
    return `${super.presentarse()} y soy un ${this.raza}`;
  }

  ladrar(): string {
    return "¡Guau!";
  }
}

const rex = new Perro("Rex", 5, "Labrador");
console.log(rex.presentarse());
// "Soy Rex, tengo 5 años y soy un Labrador"
```

Nota conceptual

La palabra clave `override` (disponible desde TypeScript 4.3) indica explícitamente que un método sobrescribe al del padre. Es obligatorio activar `noImplicitOverride` en `tsconfig.json` para detectar errores si el método padre no existe.

7.6 Clases Abstractas

Las clases abstractas definen un contrato que las subclases deben cumplir. No se pueden instanciar directamente:

```
abstract class FiguraGeometrica {
  constructor(public readonly color: string) {}

  // Método abstracto: DEBE ser implementado por subclases
  abstract calcularArea(): number;
  abstract calcularPerimetro(): number;

  // Método concreto: disponible para todas las subclases
  describir(): string {
    return `Figura ${this.color}, área: ${this.calcularArea().toFixed(2)}`;
  }
}

class Circulo extends FiguraGeometrica {
  constructor(color: string, private radio: number) {
    super(color);
  }
}
```

```
}

  calcularArea(): number {
    return Math.PI * this.radio ** 2;
  }

  calcularPerimetro(): number {
    return 2 * Math.PI * this.radio;
  }
}

// const fig = new FiguraGeometrica("rojo"); ❌ No se puede instanciar
const circulo = new Circulo("azul", 5); // ✅
console.log(circulo.describir());
```

Capítulo 8: Interfaces

8.1 ¿Qué es una Interface?

Una interface define la forma (shape) que debe tener un objeto. Es un contrato que especifica qué propiedades y métodos debe tener un objeto, sin proporcionar la implementación:

Analogía: Una interface es como un formulario de inscripción universitaria: define los campos obligatorios (nombre, edad, carrera) y opcionales (teléfono, foto). Cada estudiante llena el formulario con sus datos, pero todos siguen la misma estructura.

8.2 Declaración de Interfaces

```
// Interface básica
interface Persona {
  nombre: string;
  edad: number;
  email: string;
  telefono?: string; // Opcional
}

// Uso: el objeto DEBE cumplir con la interface
const profesor: Persona = {
  nombre: "Dr. Carlos Gómez",
  edad: 45,
  email: "carlos@humboldt.edu.co",
};

// ❌ Error: falta la propiedad 'email'
// const invalido: Persona = { nombre: "Ana", edad: 20 };
```

8.3 Interfaces con Métodos

```
interface Vehiculo {
  marca: string;
  modelo: string;
  anio: number;
  velocidadMaxima: number;

  // Métodos (solo firma, sin implementación)
  acelerar(velocidad: number): void;
  frenar(): void;
  obtenerInfo(): string;
}
```

```
// Implementación
const miAuto: Vehiculo = {
  marca: "Toyota",
  modelo: "Corolla",
  anio: 2026,
  velocidadMaxima: 200,

  acelerar(velocidad: number): void {
    console.log(`Acelerando a ${velocidad} km/h`);
  },

  frenar(): void {
    console.log("Frenando...");
  },

  obtenerInfo(): string {
    return `${this.marca} ${this.modelo} (${this.anio})`;
  },
};
```

8.4 Extensión de Interfaces

Las interfaces pueden extender otras interfaces para crear jerarquías de tipos:

```
interface EntidadBase {
  id: string;
  createdAt: Date;
  updatedAt: Date;
}

interface Usuario extends EntidadBase {
  nombre: string;
  email: string;
  rol: "admin" | "editor" | "lector";
}

interface Administrador extends Usuario {
  permisos: string[];
  nivel: number;
}

// Administrador tiene TODAS las propiedades:
// id, createdAt, updatedAt, nombre, email, rol, permisos, nivel

const admin: Administrador = {
  id: "admin-001",
```

```

    createdAt: new Date(),
    updatedAt: new Date(),
    nombre: "Laura Pérez",
    email: "laura@humboldt.edu.co",
    rol: "admin",
    permisos: ["crear", "editar", "eliminar"],
    nivel: 3,
  };

```

8.5 Interfaces vs Type Aliases

En TypeScript 2026, ambos son muy similares, pero existen diferencias prácticas importantes:

Característica	interface	type
Extensión	extends (herencia)	& (intersección)
Declaración fusionada	Sí (declaration merging)	No
Uniones	No soporta	Sí (A B)
Tipos primitivos	No	Sí (type Edad = number)
Tuplas	Limitado	Sí nativo
Uso recomendado	Contratos de objetos, APIs	Uniones, alias, utilidades

✓ Regla práctica

Usa interface cuando definas la forma de un objeto o un contrato para clases. Usa type cuando necesites uniones, intersecciones, tipos condicionales o alias de tipos primitivos. En la duda, interface es la elección más segura para objetos.

8.6 Implementación de Interfaces en Clases

```

interface Notificable {
  enviarNotificacion(mensaje: string): Promise<boolean>;
  obtenerNotificaciones(): Promise<string[]>;
}

interface Logeable {
  registrarAccion(accion: string): void;
}

// Una clase puede implementar MÚLTIPLES interfaces
class ServicioUsuario implements Notificable, Logeable {

```

```
private logs: string[] = [];  
  
async enviarNotificacion(mensaje: string): Promise<boolean> {  
    console.log(`Enviando: ${mensaje}`);  
    return true;  
}  
  
async obtenerNotificaciones(): Promise<string[]> {  
    return ["Bienvenido", "Actualización disponible"];  
}  
  
registrarAccion(accion: string): void {  
    this.logs.push(`${new Date().toISOString()} ${accion}`);  
}  
}
```

Capítulo 9: Temas Complementarios Avanzados

9.1 Genéricos (Generics)

Los genéricos permiten crear componentes reutilizables que trabajan con múltiples tipos manteniendo la seguridad de tipos. Son la herramienta más poderosa del sistema de tipos de TypeScript:

Analogía: Un genérico es como una caja universal con una etiqueta de tipo. Puedes tener Caja<Libro>, Caja<Zapatos> o Caja<Medicinas>. La caja es la misma estructura, pero el contenido está restringido al tipo que indiques en la etiqueta.

```
// Función genérica: T es un parámetro de tipo
function identidad<T>(valor: T): T {
  return valor;
}

// TypeScript infiere T automáticamente
const texto = identidad("hola");    // T = string
const numero = identidad(42);       // T = number

// Clase genérica: estructura de datos tipada
class Pila<T> {
  private elementos: T[] = [];

  push(elemento: T): void {
    this.elementos.push(elemento);
  }

  pop(): T | undefined {
    return this.elementos.pop();
  }

  peek(): T | undefined {
    return this.elementos.at(-1);
  }

  get tamano(): number {
    return this.elementos.length;
  }
}

// Uso con tipos específicos
const pilaNumeros = new Pila<number>();
pilaNumeros.push(10);
pilaNumeros.push(20);
```

```
// pilaNumeros.push("texto"); ❌ Error de tipo
```

```
const pilaTextos = new Pila<string>();  
pilaTextos.push("primero");
```

9.1.1 Restricciones en Genéricos (Constraints)

```
// Restringir T a tipos que tengan propiedad 'length'  
interface ConLongitud {  
  length: number;  
}  
  
function mostrarLongitud<T extends ConLongitud>(elemento: T): string {  
  return `Longitud: ${elemento.length}`;  
}  
  
mostrarLongitud("hola");           // ✅ string tiene length  
mostrarLongitud([1, 2, 3]);        // ✅ array tiene length  
mostrarLongitud(42);               // ❌ number no tiene length
```

9.2 Módulos e Importaciones

El sistema de módulos permite organizar el código en archivos separados con importaciones y exportaciones tipadas:

```
// — archivo: modelos/usuario.ts —  
export interface Usuario {  
  id: string;  
  nombre: string;  
  email: string;  
}  
  
export type RolUsuario = "admin" | "editor" | "lector";  
  
export function crearUsuario(nombre: string, email: string): Usuario {  
  return { id: crypto.randomUUID(), nombre, email };  
}  
  
// — archivo: servicios/auth.ts —  
import { type Usuario, type RolUsuario, crearUsuario } from "../modelos/usuario";  
  
// 'import type' solo importa tipos (se elimina en compilación)  
// Esto mejora el rendimiento del bundler
```

 Nota conceptual

En TypeScript 2026, usa siempre `import type` cuando importes solo tipos. Esto le indica al bundler (Bun) que esas importaciones no generan código JavaScript, optimizando el bundle final.

9.3 Tipos de Utilidad (Utility Types)

TypeScript incluye tipos de utilidad predefinidos que transforman tipos existentes. Son esenciales para código profesional:

Utility Type	Descripción	Ejemplo
<code>Partial<T></code>	Todas las propiedades opcionales	<code>Partial<Usuario></code>
<code>Required<T></code>	Todas las propiedades obligatorias	<code>Required<Config></code>
<code>Readonly<T></code>	Todas las propiedades de solo lectura	<code>Readonly<Estado></code>
<code>Pick<T, K></code>	Selecciona propiedades específicas	<code>Pick<Usuario, 'nombre' 'email'></code>
<code>Omit<T, K></code>	Excluye propiedades específicas	<code>Omit<Usuario, 'id'></code>
<code>Record<K, V></code>	Objeto con claves K y valores V	<code>Record<string, number></code>
<code>ReturnType<F></code>	Tipo de retorno de una función	<code>ReturnType<typeof fetch></code>
<code>Awaited<T></code>	Desenvuelve una promesa	<code>Awaited<Promise<string>></code>

```
interface Producto {
  id: string;
  nombre: string;
  precio: number;
  descripcion: string;
  stock: number;
}

// Partial: para actualizaciones parciales
function actualizarProducto(id: string, datos: Partial<Producto>): void {
  // datos puede contener cualquier subconjunto de propiedades
}

actualizarProducto("prod-001", { precio: 29.99 }); // ✅ Solo precio
actualizarProducto("prod-001", { stock: 100 });   // ✅ Solo stock

// Omit: para crear sin ID (el servidor lo genera)
type CrearProducto = Omit<Producto, "id">;

// Pick: para vistas reducidas
```

```
type ProductoResumen = Pick<Producto, "nombre" | "precio">;
```

9.4 Enums

Los enums definen un conjunto de constantes con nombre. En TypeScript 2026, se prefieren los `const` enums o los union types literales:

```
// Enum numérico
enum DiaSemana {
  Lunes = 1,
  Martes,    // 2
  Miercoles, // 3
  Jueves,    // 4
  Viernes,   // 5
  Sabado,    // 6
  Domingo,   // 7
}

const hoy: DiaSemana = DiaSemana.Lunes;

// Enum de strings (más legible en depuración)
enum EstadoPedido {
  Pendiente = "PENDIENTE",
  EnProceso = "EN_PROCESO",
  Enviado = "ENVIADO",
  Entregado = "ENTREGADO",
  Cancelado = "CANCELADO",
}

// ✅ Alternativa moderna (preferida en 2026): union literal
type Estado = "pendiente" | "en_proceso" | "enviado" | "entregado";
```

✓ Consejo profesional

En proyectos nuevos de 2026, se prefieren los union types literales sobre los enums clásicos. Los union literals no generan código JavaScript adicional (son solo tipos) y funcionan mejor con tree-shaking de los bundlers.

9.5 Manejo de Errores Tipado

TypeScript 2026 promueve el manejo de errores tipado y predecible, siguiendo el patrón `Result`:

```
// Patrón Result: manejar errores sin excepciones
type Result<T, E = Error> =
```

```
| { success: true; data: T }
| { success: false; error: E };

// Función que retorna Result en vez de lanzar excepciones
function dividir(a: number, b: number): Result<number, string> {
  if (b === 0) {
    return { success: false, error: "División por cero" };
  }
  return { success: true, data: a / b };
}

// Uso seguro del resultado
const resultado = dividir(10, 3);

if (resultado.success) {
  console.log(`Resultado: ${resultado.data}`); // TypeScript sabe que .data existe
} else {
  console.error(`Error: ${resultado.error}`); // TypeScript sabe que .error
  existe
}
```

9.6 Programación Asincrónica Tipada

El manejo de operaciones asincrónicas es fundamental en aplicaciones modernas. TypeScript tipifica completamente las promesas y `async/await`:

```
// Función asincrónica con tipo de retorno explícito
interface DatosAPI {
  usuarios: { id: number; nombre: string }[];
  total: number;
}

async function obtenerDatos(url: string): Promise<DatosAPI> {
  const respuesta = await fetch(url);

  if (!respuesta.ok) {
    throw new Error(`HTTP ${respuesta.status}`);
  }

  const datos: DatosAPI = await respuesta.json();
  return datos;
}

// Uso con manejo de errores
async function main(): Promise<void> {
  try {
    const datos = await obtenerDatos("https://api.ejemplo.com/users");
  }
}
```

```
    console.log(`Total de usuarios: ${datos.total}`);

    for (const usuario of datos.usuarios) {
        console.log(`- ${usuario.nombre}`);
    }
} catch (error) {
    if (error instanceof Error) {
        console.error(`Error: ${error.message}`);
    }
}
}
```

9.7 Decoradores (TypeScript 5.x+ Estándar)

Los decoradores estándar de ECMAScript (Stage 3, soportados nativamente en TypeScript 5.x+) permiten modificar el comportamiento de clases y sus miembros de forma declarativa:

```
// Decorador de método: medir tiempo de ejecución
function medirTiempo<T extends (...args: any[]) => any>(
    target: T,
    context: ClassMethodDecoratorContext
) {
    return function (this: any, ...args: Parameters<T>): ReturnType<T> {
        const inicio = performance.now();
        const resultado = target.apply(this, args);
        const fin = performance.now();
        console.log(`${String(context.name)}: ${(fin - inicio).toFixed(2)}ms`);
        return resultado;
    };
}

class ServicioDatos {
    @medirTiempo
    procesarRegistros(datos: number[]): number {
        return datos.reduce((sum, n) => sum + n, 0);
    }
}
```

9.8 Validación con Zod

Zod es la librería de validación de esquemas más utilizada en el ecosistema TypeScript 2026. Permite definir esquemas que validan datos en runtime e infieren tipos automáticamente:

```
import { z } from "zod";
```

```
// Definir un esquema de validación
const EsquemaEstudiante = z.object({
  nombre: z.string().min(2).max(100),
  edad: z.number().int().min(16).max(100),
  email: z.string().email(),
  carrera: z.enum(["ingenieria", "medicina", "derecho"]),
  promedio: z.number().min(0).max(5).optional(),
});

// Inferir el tipo TypeScript automáticamente del esquema
type Estudiante = z.infer<typeof EsquemaEstudiante>;

// Validar datos (por ejemplo, de un formulario o API)
const resultado = EsquemaEstudiante.safeParse({
  nombre: "Ana",
  edad: 22,
  email: "ana@humboldt.edu.co",
  carrera: "ingenieria",
});

if (resultado.success) {
  console.log(resultado.data); // Tipo: Estudiante
} else {
  console.error(resultado.error.issues);
}
```

Capítulo 10: Proyecto Práctico Integrador

10.1 Descripción del Proyecto

Para consolidar todos los conceptos aprendidos, construiremos paso a paso un Sistema de Gestión Académica con TypeScript y Bun. Este proyecto integra tipado, interfaces, clases, genéricos, funciones flecha y manejo de errores:

10.2 Definición de Tipos e Interfaces

```
// — types/academico.ts —

// Enums como union literals (práctica moderna)
type Carrera = "ingenieria_sistemas" | "ingenieria_industrial"
  | "administracion" | "derecho" | "medicina";

type EstadoMatricula = "activa" | "suspendida" | "graduado" | "retirado";

// Interfaces base
interface EntidadBase {
  readonly id: string;
  createdAt: Date;
  updatedAt: Date;
}

interface Persona extends EntidadBase {
  nombre: string;
  apellido: string;
  email: string;
  documento: string;
}

interface Estudiante extends Persona {
  matricula: string;
  carrera: Carrera;
  semestre: number;
  estado: EstadoMatricula;
  promedio: number;
}

interface Materia {
  codigo: string;
  nombre: string;
  creditos: number;
  profesor: string;
}
```

```
interface Inscripcion {
  estudianteId: string;
  materiaId: string;
  nota?: number;
  periodo: string;
}
```

10.3 Clase Genérica de Repositorio

```
// — repositorios/repositorio.ts —

type Result<T, E = string> =
  | { ok: true; valor: T }
  | { ok: false; error: E };

class Repositorio<T extends EntidadBase> {
  private datos: Map<string, T> = new Map();

  agregar(entidad: T): Result<T> {
    if (this.datos.has(entidad.id)) {
      return { ok: false, error: `ID ${entidad.id} ya existe` };
    }
    this.datos.set(entidad.id, { ...entidad, updatedAt: new Date() });
    return { ok: true, valor: entidad };
  }

  obtener(id: string): Result<T> {
    const entidad = this.datos.get(id);
    if (!entidad) {
      return { ok: false, error: `Entidad ${id} no encontrada` };
    }
    return { ok: true, valor: entidad };
  }

  actualizar(id: string, datos: Partial<Omit<T, 'id'>>): Result<T> {
    const existente = this.datos.get(id);
    if (!existente) {
      return { ok: false, error: `Entidad ${id} no encontrada` };
    }
    const actualizado = { ...existente, ...datos, updatedAt: new Date() } as T;
    this.datos.set(id, actualizado);
    return { ok: true, valor: actualizado };
  }

  listar(): T[] {
    return Array.from(this.datos.values());
  }
}
```

```
}

filtrar(predicado: (entidad: T) => boolean): T[] {
  return this.listar().filter(predicado);
}

get cantidad(): number {
  return this.datos.size;
}
}
```

10.4 Servicio Académico

```
// — servicios/academico.ts —

class ServicioAcademico {
  private repoEstudiantes = new Repositorio<Estudiante>();
  private repoMaterias = new Repositorio<Materia & EntidadBase>();

  registrarEstudiante(
    datos: Omit<Estudiante, 'id' | 'createdAt' | 'updatedAt' | 'promedio'>
  ): Result<Estudiante> {
    const estudiante: Estudiante = {
      ...datos,
      id: crypto.randomUUID(),
      createdAt: new Date(),
      updatedAt: new Date(),
      promedio: 0,
    };
    return this.repoEstudiantes.agregar(estudiante);
  }

  obtenerMejoresEstudiantes(minPromedio: number = 4.0): Estudiante[] {
    return this.repoEstudiantes
      .filtrar((e) => e.promedio >= minPromedio && e.estado === "activa")
      .sort((a, b) => b.promedio - a.promedio);
  }

  generarReporte(): string {
    const estudiantes = this.repoEstudiantes.listar();
    const activos = estudiantes.filter((e) => e.estado === "activa");
    const promedioGeneral = activos.length > 0
      ? activos.reduce((sum, e) => sum + e.promedio, 0) / activos.length
      : 0;

    return [
      `Total estudiantes: ${estudiantes.length}`,
    ]
  }
}
```

```
    `Estudiantes activos: ${activos.length}`,\n    `Promedio general: ${promedioGeneral.toFixed(2)}`,\n  ].join("\\n");\n  }\n}
```

10.5 Ejecución del Proyecto

```
// — index.ts —\n\nconst servicio = new ServicioAcademico();\n\n// Registrar estudiantes\nconst resultado = servicio.registrarEstudiante({\n  nombre: "Ana María",\n  apellido: "Gómez López",\n  email: "ana.gomez@humboldt.edu.co",\n  documento: "1234567890",\n  matricula: "2026-001",\n  carrera: "ingenieria_sistemas",\n  semestre: 3,\n  estado: "activa",\n});\n\nif (resultado.ok) {\n  console.log(`Estudiante registrado: ${resultado.valor.nombre}`);\n} else {\n  console.error(`Error: ${resultado.error}`);\n}\n\n// Ejecutar con Bun\n// bun run index.ts
```

✓ Ejercicio final

Extiende el proyecto añadiendo: (1) una interfaz para profesores, (2) un método para inscribir materias con validación de prerrequisitos, (3) un sistema de calificaciones que actualice el promedio automáticamente.

Glosario de Términos

Término	Definición
Inferencia de tipos	Capacidad de TypeScript para deducir automáticamente el tipo de una variable
Type guard	Expresión condicional que refina el tipo de una variable en un bloque de código
Union type	Tipo que permite múltiples tipos posibles, separados por
Intersection type	Tipo que combina múltiples tipos en uno, usando &
Generic	Parámetro de tipo que permite crear componentes reutilizables
Discriminated union	Unión donde cada variante tiene una propiedad discriminante común
Utility type	Tipo predefinido que transforma otros tipos (Partial, Pick, Omit, etc.)
Type alias	Nombre personalizado para un tipo, definido con la palabra clave type
Interface	Contrato que define la forma de un objeto
Decorator	Función que modifica el comportamiento de clases o sus miembros
Arrow function	Función con sintaxis concisa (=>) que hereda el contexto this
Tuple	Array con longitud fija y tipos definidos por posición
Runtime	Entorno de ejecución de código (Bun, Node.js, Deno)
Bundler	Herramienta que empaqueta múltiples archivos en uno optimizado
Tree-shaking	Eliminación de código muerto durante el empaquetado

Bibliografía y Recursos

1. *TypeScript Official Documentation* — <https://www.typescriptlang.org/docs/>
2. *Bun Official Documentation* — <https://bun.sh/docs>
3. *TypeScript Deep Dive (Basarat Ali Syed)* — <https://basarat.gitbook.io/typescript>
4. *Zod Documentation* — <https://zod.dev>
5. *Biome Documentation* — <https://biomejs.dev>
6. *Matt Pocock — Total TypeScript (2024-2026)*
7. *ECMAScript Specification* — <https://tc39.es/ecma262/>
8. *Hono Framework* — <https://hono.dev>

© 2026 — Universidad Alexander von Humboldt
Material didáctico de uso académico